MEASI INSTITUTE OF INFORMATION TECHNOLOGY

(Approved by AICTE & Affiliated to University of Madras)
CHENNAI – 600 014

MEASI
Institute *of*
Information
Technology

MASTER OF COMPUTER APPLICATIONS

ACADEMIC YEAR 2026-2027
SEMESTER – III

Practical Record

ADVANCED MACHINE LEARNING LAB (535S3A)

REG. NO : _____

NAME : _____

BATCH : _____

MEASI INSTITUTE OF INFORMATION TECHNOLOGY
(Approved by AICTE & Affiliated to University of Madras)
CHENNAI- 600 014

MCA PRACTICALS

ADVANCED MACHINE LEARNING LAB

Academic Year 2026-2027
Semester - III

NAME : **CLASS :**

REG-NO : **BATCH :**

This is to certify that this is the bonafide record of work done in the Computer Science Laboratory of MEASI Institute of Information Technology, submitted for the University of Madras Practical Examination held on _____ at MEASI Institute of Information Technology, Chennai-600 014.

STAFF IN-CHARGE **HEAD OF THE DEPARTMENT**

INTERNAL EXAMINER **EXTERNAL EXAMINER**

DATE: _____

INDEX

S.No.	Date	Program Name	Page No.	Signature
1(a)	07.07.2026	Central Tendency Measures: Mean, Median, Mode, Measure of Dispersion: Variance, Standard Deviation Using Library Functions	01	
1(b)	07.07.2026	Central Tendency Measures: Mean, Median, Mode, Measure of Dispersion: Variance, Standard Deviation Using Library Functions using Mathematical Formulas	05	
2.1	14.07.2026	Simple Linear Regression	09	
2.2(a)	16.07.2026	Multiple Linear Regression	13	
2.2(b)	21.07.2026	Multiple Linear Regression Using Label Encoders	17	
3	23.07.2026	Logistic Regression	24	
4	04.08.2026	Binary Classification Using Logistic Regression	30	
5(a)	11.08.2026	K-Nearest Neighbors Algorithm	35	
5(b)	13.08.2026	Naive Bayes Classification	39	
6	20.08.2026	Decision Tree with Hyperparameter Tuning	44	
7(a)	27.08.2026	K-Means Clustering with Mathematical Formulas	55	
7(b)	03.09.2026	K-Means Clustering using Python Library	60	
8	15.09.2026	CNN Using Tensorflow	63	
9	22.09.2026	Auto Encoders Using Tensorflow	67	
10	06.10.2026	Long Short-Term Memory Using Tensorflow	73	

Ex. No: 01 (a) Date:	**Central Tendency Measures: Mean, Median, Mode, Measure of Dispersion: Variance, Standard Deviation Using Library Functions**	

AIM:

To read and analyze a CSV file using Pandas and find the mean, median, mode, variance and standard deviation of marks.

ALGORITHM:

Step 1: Import the Pandas library.

Step 2: Read the CSV file using pd.read_csv().

Step 3: Display the dataset and show the first 3 rows.

Step 4: Display the column names and shape of the dataset.

Step 5: Display information about the dataset.

Step 6: Separate and display numerical and categorical data.

Step 7: Check for missing values in the dataset.

Step 8: Select the marks column from the dataset.

Step 9: Calculate the mean, median and mode of the marks.

Step 10: Calculate the variance and standard deviation of the marks.

Step 11: Display all the calculated results.

PROGRAM:

```
import pandas as pd
data = pd.read_csv(r"C:\Users\shaji\OneDrive\Pictures\3rd Sem\aml\demo\csv01.csv")


print(data)
print(data.head(3))
print(data.columns)
print(data.shape)
data.info()


print(data.select_dtypes(include="number"))
print(data.select_dtypes(include="object"))
print(data.isnull())
print(data.isnull().sum())


marks = data["marks"]


mean = marks.mean()
median = marks.median()
mode = marks.mode()
variance = marks.var()
std = marks.std()
```

```
("\n Central Tendency measures")


print("mean", mean)

print("median", median)

print("mode", mode)


print("\n measures of dispersion")


print("variance", variance)

print("Standard division:", std)
```

OUTPUT:

```
   student_id     name  marks department
0           1     Arun     75         CS
1           2     Bala     82         IT
2           3   Charan     90         CS
3           4    Divya     68         IT
4           5     Esha     82         CS
5           6  Fathima     55         IT
6           7    Gokul     75         CS
7           8     Hari     95         IT
8           9     Isha     60         CS
9          10    Jeeva     82         IT
   student_id    name  marks department
0           1    Arun     75         CS
1           2    Bala     82         IT
2           3  Charan     90         CS
Index(['student_id', 'name', 'marks', 'department'], dtype='object')
(10, 4)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 4 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   student_id  10 non-null     int64
 1   name        10 non-null     object
 2   marks       10 non-null     int64
 3   department  10 non-null     object
dtypes: int64(2), object(2)
memory usage: 452.0+ bytes
   student_id  marks
0           1     75
1           2     82
2           3     90
3           4     68
4           5     82
5           6     55
6           7     75
7           8     95
8           9     60
9          10     82
```

```
   student_id   name  marks  department
0       False  False  False       False
1       False  False  False       False
2       False  False  False       False
3       False  False  False       False
4       False  False  False       False
5       False  False  False       False
6       False  False  False       False
7       False  False  False       False
8       False  False  False       False
9       False  False  False       False
student_id    0
name          0
marks         0
department    0
dtype: int64
mean 76.4
median 78.5
mode 0    82
Name: marks, dtype: int64

 measures of dispersion
variance 158.4888888888889
Standard division: 12.58923702568543
PS C:\xampp\htdocs\hmsm>
```

RESULT:

Thus, the given program was successfully implemented using Python, and the required output was obtained successfully.

Ex. No: 01 (b) Date:	**Central Tendency Measures: Mean, Median, Mode, Measure of Dispersion: Variance, Standard Deviation Using Library Functions using Mathematical Formulas**	

AIM:

To calculate the mean, median, mode, variance and standard deviation of given marks using Python.

ALGORITHM:

Step 1: Get the number of observations from the user.

Step 2: Read the marks and store them in a list.

Step 3: Calculate the mean by adding all values and dividing by the number of observations.

Step 4: Sort the marks in ascending order.

Step 5: Calculate the median based on whether the number of observations is odd or even.

Step 6: Find the mode by counting the frequency of each value.

Step 7: Calculate the sum of squared differences between each value and the mean.

Step 8: Calculate the variance and standard deviation.

Step 9: Display the mean, median, mode, variance and standard deviation.

PROGRAM:

```
n=int(input("Enter number of odservation:"))
marks=[]
for i in range(n):
    value=int(input("enter the value:"))
    marks.append(value)
print("given Data:",marks)


#! Calculate The Mean
total=0
for x in marks:
    total+=x
    mean=total/n


#! Calculate the Median
marks.sort()
if n%2==0:
    median=(marks[n//2-1]+marks[n//2])
else:
    median=marks[n//2]


#! Calculate The Mode
max_count = 0
modes = []
```

```
for x in marks:
    count = 0
    for y in marks:
        if x == y:
            count += 1
    if count > max_count:
        max_count = count
        modes = [x]
    elif count == max_count and x not in modes:
        modes.append(x)

#! Calculate Sum Difference and Standard Divition
sum_difference=0
for x in marks:
    sum_difference+=(x-mean)**2
variance=sum_difference/(n-2)
standard_divition=variance**0.5

#! Print Every Result
print("\nMean=",mean)
print("Median=",median)
print("Mode =", modes, "Count =", max_count)
print("Variance=",variance)
print("Standard Devition=",standard_divition)
```

OUTPUT:



RESULT:

Thus, the given program was successfully implemented using Python, and the required output was obtained successfully.

Ex. No: 2.1 Date:	Simple Linear Regression	

AIM:

To develop a Simple Linear Regression model using Python to predict salary based on years of experience.

ALGORITHM:

Step 1: Import Pandas, Matplotlib and Linear Regression libraries.

Step 2: Read the salary dataset from the CSV file.

Step 3: Select Experience as input (X) and Salary as output (y).

Step 4: Create a Linear Regression model.

Step 5: Train the model using the experience and salary data.

Step 6: Predict the salary for the existing data.

Step 7: Display the intercept and coefficient of the model.

Step 8: Get years of experience from the user.

Step 9: Predict the salary for the given experience.

Step 10: Draw a scatter plot and regression line using Matplotlib.

Step 11: Display the predicted salary and graph.

PROGRAM:

```
import pandas as pd

import matplotlib.pyplot as plt

from sklearn.linear_model import LinearRegression

data = pd.read_csv(r"C:\Users\shaji\OneDrive\Pictures\3rd Sem\aml\demo\experience_salary.csv")

print("Dataset")

print(data)

print(data.columns)


# Input and Output

X = data[["Experience"]]

y = data["Salary"]


# Create Linear Regression model

model = LinearRegression()

model.fit(X, y)

y_pred = model.predict(X)

print("Intercept =", model.intercept_)

print("\nCoefficient =", model.coef_[0])


# Get input from user

experience = float(input("\nEnter Years of Experience: "))

new_data = pd.DataFrame({
```

```
    "Experience": [experience]
})
prediction = model.predict(new_data)
print("Predicted Salary =", prediction[0])


#The Graph
plt.Figure()
plt.scatter(X["Experience"], y, label="Actual Salary")
plt.plot(X["Experience"], y_pred, label="Regression Line")
plt.xlabel("Years of Experience")
plt.ylabel("Salary")
plt.title("Experience vs Salary - Linear Regression")
plt.legend()
plt.show()
```

OUTPUT:

```
Dataset
   Experience  Salary
0           1   30000
1           2   35000
2           3   40000
3           4   45000
4           5   52000
5           6   58000
6           7   65000
7           8   72000
8           9   80000
9          10   90000
Index(['Experience', 'Salary'], dtype='object')
Intercept = 20666.666666666664

Coefficient = 6551.515151515152

Enter Years of Experience: 9
Predicted Salary = 79630.30303030304
```



RESULT:

Thus, the given program was successfully implemented using Python, and the required output was obtained successfully.

Ex. No: 2.2 (a) Date:	Multiple Linear Regression	

AIM:

To develop a Multiple Linear Regression model using Python to predict employee salary based on experience, test score and interview score.

ALGORITHM:

Step 1: Import the required libraries such as Pandas, NumPy, Matplotlib and Scikit-learn.

Step 2: Read the employee salary dataset from the CSV file.

Step 3: Convert experience values from words into numbers.

Step 4: Fill missing experience values with 0 and missing test scores with the median.

Step 5: Select experience, test score and interview score as input features.

Step 6: Select salary as the output value.

Step 7: Create and train the Multiple Linear Regression model.

Step 8: Display the intercept and coefficients of each feature.

Step 9: Predict salaries and calculate residuals.

Step 10: Calculate MSE, RMSE and R^2 score to evaluate the model.

Step 11: Predict the salary of a new employee using given details.

Step 12: Draw the Actual vs Predicted Salary graph.

PROGRAM:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as pt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error,r2_score

df=pd.read_csv(r"C:\Users\shaji\OneDrive\Pictures\3rd Sem\aml\demo\employee_salary.csv")

e_m={"zero":0,"one":1,"two":2,"three":3,"four":4,"five":5,"six":6,"seven":7,"eight":8,
"nine":9,"ten":10,"eleven":11}

df["experience"]=df["experience"].map(e_m)
df["experience"]=df["experience"].fillna(0)
median=df["test_score(out of 10)"].median()
df["test_score(out of 10)"]=df["test_score(out of 10)"].fillna(median)
x=df[["experience","test_score(out of 10)","interview_score(out of 10)"]]
y=df["salary($)"]
model=LinearRegression()
model.fit(x,y)
print("intercept:",model.intercept_)
for feature,coefficint in zip(x.columns,model.coef_):
    print(feature,"=",coefficint)
y_pred=model.predict(x)
```

```
df["predict salary"]=y_pred
df["residual"]=df["salary($)"]-df["predict salary"]
print(df)


mse=mean_squared_error(y,y_pred)
rmse=np.sqrt(mse)
r2=r2_score(y,y_pred)
print(f"mse:{mse} - rmse:{rmse} - r2_score:{r2}")

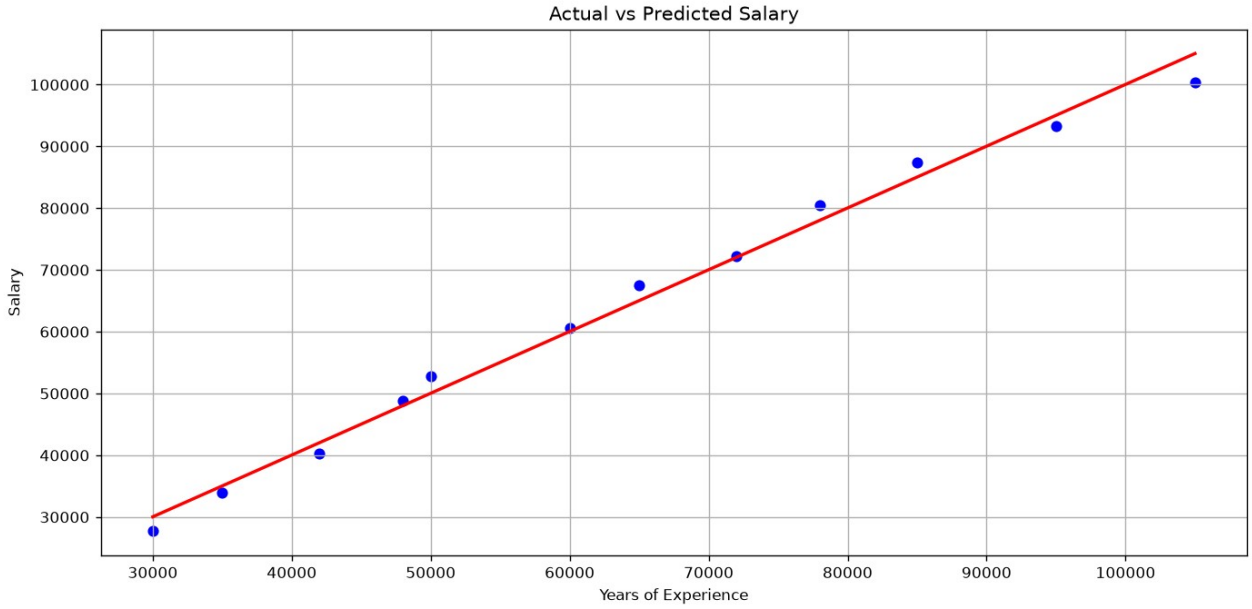

new_employee = pd.DataFrame({
"experience":[6],
"test_score(out of 10)":[9],
"interview_score(out of 10)":[8]
})
salary=model.predict(new_employee)
print("\n Predicted Salary:",salary[0])


pt.figure(figsize=(8,6))
pt.scatter(y,y_pred,color="blue")
pt.plot([y.min(),y.max()],[y.min(),y.max()],color="red",linewidth=2)
pt.grid(True)
pt.title("Actual vs Predicted Salary")
pt.xlabel("Years of Experience")
pt.ylabel("Salary")
pt.show()
```

OUTPUT:

```
intercept: 30619.63959081367
experience = 6629.428900480995
test_score(out of 10) = 770.7811123907583
interview_score(out of 10) = -1131.5290291985664
    experience  test_score(out of 10)  interview_score(out of 10)  salary($)  predict salary     residual
0            0                      5                           6      30000    27684.370978  2315.629022
1            1                      6                           7      35000    33953.051961  1046.948039
2            2                      7                           8      42000    40221.732945  1778.267055
3            3                      8                           7      48000    48753.471987  -753.471987
4            4                      6                           8      50000    52709.809633 -2709.809633
5            5                      9                           9      60000    60520.052842  -520.052842
6            6                      8                           8      65000    67510.229659 -2510.229659
7            7                      7                           9      72000    72237.348418  -237.348418
8            8                      9                           9      78000    80408.339543 -2408.339543
9            9                      8                           8      85000    87398.516361 -2398.516361
10          10                     10                          10      95000    93306.449428  1693.550572
11          11                      9                           9     105000   100296.626245  4703.373755
mse:5058510.037712004 - rmse:2249.1131669420292 - r2_score:0.9903528752906283

 Predicted Salary: 68281.01077162793
```



RESULT:

Thus, the given program was successfully implemented using Python, and the required output was obtained successfully.

Ex. No: 2.2 (b) Date:	**Multiple Linear Regression Using Label Encoder**	

AIM:

To predict job salary using Multiple Linear Regression and Label Encoding in Python.

ALGORITHM:

Step 1: Import the required libraries such as Pandas, NumPy, Matplotlib and Scikit-learn.

Step 2: Read the job salary dataset from the CSV file.

Step 3: Convert categorical data into numerical values using Label Encoding.

Step 4: Separate the input features (X) and salary as the output (y).

Step 5: Split the dataset into training and testing data using train_test_split().

Step 6: Create a Linear Regression model.

Step 7: Train the model using the training data.

Step 8: Display the intercept and coefficients of the model.

Step 9: Predict salary using the testing data.

Step 10: Calculate MSE, RMSE and R^2 score to evaluate the model.

Step 11: Compare actual salary and predicted salary for one test record.

Step 12: Create a comparison table containing actual salary, predicted salary and residual.

Step 13: Draw the Actual vs Predicted Salary graph using Matplotlib.

PROGRAM:

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

from sklearn.preprocessing import LabelEncoder

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LinearRegression

from sklearn.metrics import mean_squared_error, r2_score


# Step 1: Read Dataset

df = pd.read_csv(r"C:\Users\shaji\OneDrive\Pictures\3rd Sem\aml\demo\job_salary_prediction_dataset (1).csv")

print(df.head())


# Step 2: Label Encoding

le = LabelEncoder()

df["job_title"] = le.fit_transform(df["job_title"])

df["education_level"] = le.fit_transform(df["education_level"])

df["industry"] = le.fit_transform(df["industry"])

df["company_size"] = le.fit_transform(df["company_size"])

df["location"] = le.fit_transform(df["location"])

df["remote_work"] = le.fit_transform(df["remote_work"])


# Step 3: Independent and Dependent Variables

X = df.drop("salary", axis=1)
```

```
y = df["salary"]


# Step 4: Split Dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)
print("\nTraining Records :", len(X_train))
print("Testing Records :", len(X_test))


# Step 5: Create Model
model = LinearRegression()


# Step 6: Train Model
model.fit(X_train, y_train)


# Step 7: Intercept and Coefficients
print("\nIntercept =", model.intercept_)


print("\nCoefficients")
for feature, coef in zip(X.columns, model.coef_):
    print(feature, "=", coef)
```

```
# Step 8: Predict Test Data
y_pred = model.predict(X_test)

# Step 9: Model Evaluation
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("\nMean Squared Error =", mse)
print("Root Mean Squared Error =", rmse)
print("R2 Score =", r2)

# Step 10: Compare One Test Record
print("\nFirst Record from Test Dataset")
print(X_test.iloc[0])

print("\nActual Salary =", y_test.iloc[0])
print("Predicted Salary =", y_pred[0])
print("Residual =", y_test.iloc[0] - y_pred[0])

# Step 11: Comparison Table
comparison = pd.DataFrame({
    "Actual Salary": y_test.values,
    "Predicted Salary": y_pred,
    "Residual": y_test.values - y_pred
```

```
})


print("\nComparison Table")
print(comparison.head())


# Step 12: Graph
plt.figure(figsize=(8, 6))
plt.scatter(y_test, y_pred)
plt.plot(
    [y_test.min(), y_test.max()],
    [y_test.min(), y_test.max()],
    color="red"
)
plt.title("Actual vs Predicted Salary")
plt.xlabel("Years of Experience")
plt.ylabel("Salary")
plt.grid(True)
plt.show()
```

OUTPUT:

```
                job_title education_level       industry company_size       location remote_work  experience_years  salary
0           Project Manager       Master's        Finance        Large        Chicago         No                 4  113033
1  Machine Learning Engineer      Bachelor's         Retail        Small         Austin        Yes                 6   96991
2     Database Administrator            PhD  Manufacturing        Small         Boston         No                15  165819
3           Business Analyst            PhD     Technology        Small  San Francisco         No                 8  149852
4           Project Manager       Master's     Technology        Large         Austin        Yes                 6  148281

Training Records : 80
Testing Records : 20

Intercept = 87760.0922342502

Coefficients
Job_title = 460.3209830969573
Education_level = 13205.294063323849
Industry = 1100.48932316243
Company_size = -11409.04857541191
Location = -313.3470589589451
Remote_work = 2077.28980479447
```

```
Mean Squared Error = 161007845.50326848
Root Mean Squared Error = 12688.886692821734
R2 Score = 0.814889336048958

First Record from Test Dataset
job_title           3
education_level     1
industry            4
company_size        1
location            2
remote_work         0
experience_years    8
Name: 83, dtype: int64

Actual Salary = 160197
Predicted Salary = 137822.16679481056
Residual = 22374.83320518944

Comparison Table
   Actual Salary  Predicted Salary      Residual
0         160197     137822.166795  22374.833205
1          95508     115859.535435 -20351.535435
2         177041     165507.033938  11533.966062
3         194615     186580.258225   8034.741775
4         144925     142043.344672   2881.655328
```



RESULT:

Thus, the given program was successfully implemented using Python, and the required output was obtained successfully.

Ex. No: 03 Date:	Logistic Regression	

AIM:

To develop a Logistic Regression model using Python to predict whether a student will pass or fail based on marks, assignment and attendance.

ALGORITHM:

Step 1: Import the required libraries such as Pandas, NumPy, Matplotlib and Scikit-learn.

Step 2: Read the student pass/fail dataset from the CSV file.

Step 3: Select mark, assignment and attendance as input features (X).

Step 4: Select pass_fail as the output (y).

Step 5: Split the dataset into training and testing data.

Step 6: Create and train the Logistic Regression model.

Step 7: Predict the pass or fail result using the test data.

Step 8: Calculate the accuracy and confusion matrix.

Step 9: Display the intercept and coefficients of the model.

Step 10: Get marks, assignment and attendance from the user.

Step 11: Calculate the score and pass probability using the sigmoid function.

Step 12: Predict whether the new student will PASS or FAIL.

Step 13: Display the probability graph using Matplotlib.

PROGRAM:

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as pt

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LogisticRegression

from sklearn.metrics import accuracy_score, confusion_matrix


df = pd.read_csv(r"C:\Users\shaji\OneDrive\Pictures\3rd Sem\aml\demo\student_pass_fail_500_rows.csv")

print("Sample values:")

print(df)


x = df[["mark", "assignment", "attendance"]]

y = df["pass_fail"]

x_train, x_test, y_train, y_test = train_test_split(

    x,

    y,

    test_size=0.4,

    random_state=42

)


model = LogisticRegression()


model.fit(x_train, y_train)
```

```
y_pred = model.predict(x_test)
accuracy = accuracy_score(y_test, y_pred)
print("\nAccuracy:", accuracy)

cm = confusion_matrix(y_test, y_pred)
print("\nConfusion matrix:")
print(cm)

print("\nLogistic regression equation:")

print("Intercept:", model.intercept_)

print("Coefficients:")
print(model.coef_)

# New student prediction

mark = float(input("Enter mark (0-100): "))
assignment = float(input("Enter assignment mark (0-10): "))
attendance = float(input("Enter attendance (0-5): "))

new_student = pd.DataFrame({
    "mark":[mark],
    "assignment":[assignment],
    "attendance":[attendance] })
```

```
z = (
    model.intercept_[0]
    + model.coef_[0][0] * mark
    + model.coef_[0][1] * assignment
    + model.coef_[0][2] * attendance
)

sigmoid = 1 / (1 + np.exp(-z))

print("Score:", z)

print("Estimated pass probability:", sigmoid)

prediction = model.predict(new_student)
probability = model.predict_proba(new_student)

if prediction[0] == 1:
    print("\nPrediction: PASS")
else:
    print("\nPrediction: FAIL")

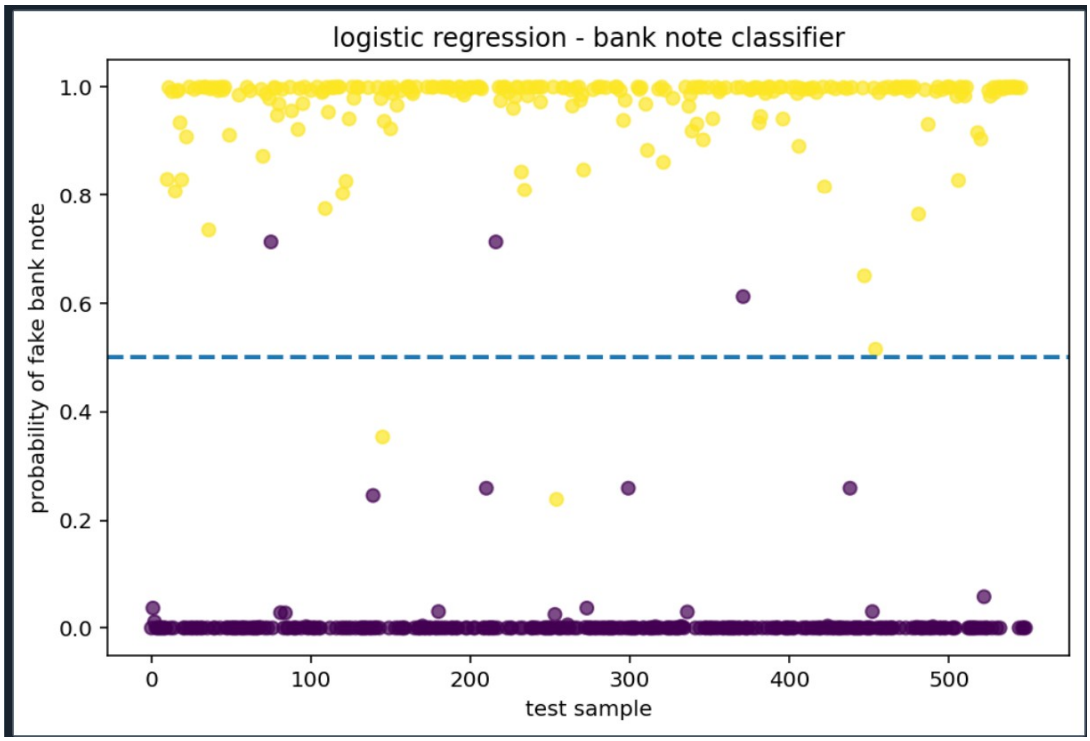
pt.figure()
pt.scatter(
    range(len(y_test)),
    model.predict_proba(x_test)[:, 1],
```

```
    c=y_test,
    alpha=0.7
)
pt.axhline(
    y=0.5,
    linestyle="--",
    linewidth=2
)
pt.xlabel("test sample")
pt.ylabel("probability of Student got pass")
pt.title("logistic regression - Students")
pt.show()
```

OUTPUT:

```
Sample values:
     mark  assignment  attendance  pass_fail
0      27           1           4          0
1      82           7           4          1
2      73          10           3          1
3      55           9           5          1
4      55           3           0          0
..    ...         ...         ...        ...
495    53           6           2          0
496    29           5           3          0
497    50           3           4          0
498    95          10           2          0
499    54           5           5          1

[500 rows x 4 columns]
Accuracy: 0.905
Confusion matrix:
[[152   7]
 [ 12  29]]
Logistic regression equation:
Intercept: [-13.04023972]
Coefficients:
[[0.05721949 0.51702483 1.46898507]]
Enter mark (0-100): 45
Enter assignment mark (0-10): 4
Enter attendance (0-5): 2
Score: -5.459293010826635
Estimated pass probability: 0.004238522483633748
Prediction: FAIL
```



RESULT:

Thus, the given program was successfully implemented using Python, and the required output was obtained successfully.

Ex. No: 04 Date:	Binary Classification	

AIM:

To perform binary classification using Logistic Regression in Python to classify banknotes into two classes based on their features.

ALGORITHM:

Step 1: Import the required libraries.

Step 2: Read the banknote dataset from the CSV file.

Step 3: Select variance, skewness, curtosis and entropy as input features.

Step 4: Select Class as the target variable with two categories.

Step 5: Split the dataset into training and testing data.

Step 6: Create and train the Logistic Regression model.

Step 7: Get new banknote details from the user.

Step 8: Calculate the classification score and sigmoid probability.

Step 9: Predict the banknote class using the trained model.

Step 10: Display the class probabilities.

Step 11: Plot the classification probability graph.

PROGRAM:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as pt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression


df = pd.read_csv(r"C:\Users\shaji\OneDrive\Pictures\3rd Sem\aml\demo\banknote_authentication_500_rows.csv")
print(df)


x = df[["Variance", "Skewness", "Curtosis", "Entropy"]]
y = df["Class"]


x_train, x_test, y_train, y_test = train_test_split(
    x,
    y,
    test_size=0.4,
    random_state=42
)
model = LogisticRegression()
model.fit(x_train, y_train)
y_pred = model.predict(x_test)
print("\nIntercept:", model.intercept_)
print("\nCoefficient:", model.coef_)
```

```
print()
print("Enter the new details:")
print("----------------------")
v = float(input("Enter variance:"))
s = float(input("Enter skewness:"))
c = float(input("Enter curtosis:"))
e = float(input("Enter entropy:"))
new = pd.DataFrame({
    "Variance": [v],
    "Skewness": [s],
    "Curtosis": [c],
    "Entropy": [e]
})
xyz = ( model.intercept_[0]+ model.coef_[0][0] * v+ model.coef_[0][1] * s+ model.coef_[0][2]
* c+ model.coef_[0][3] * e)
print()
print("values", xyz)
sigmoid = 1 / (1 + np.exp(-xyz))
print("Fake Note Probability:", sigmoid)
pre = model.predict(new)
pro = model.predict_proba(new)
if pre[0] == 1:
    print("\nZERO NOTE")
```

```
else:
    print("\nFAKE NOTE")
print("Probability of FAKE NOTE:", pro[0][0])
print("Probability of ZERO NOTE:", pro[0][1])


pt.figure()
pt.scatter(
    range(len(y_test)),
    model.predict_proba(x_test)[:, 1],
    c=y_test,
    alpha=0.7
)

pt.axhline(
    y=0.5,
    linestyle="--",
    linewidth=2
)

pt.xlabel("test sample")
pt.ylabel("probability of fake bank note")
pt.title("logistic regression - banknote")
pt.show()
```

OUTPUT:

```
    Variance  Skewness  Curtosis  Entropy  Class
0     3.6216    8.6661   -2.8073 -0.44699      0
1     4.5459    8.1674   -2.4586 -1.46210      0
2     3.8660   -2.6383    1.9242  0.10645      0
3     3.4566    9.5228   -4.0112 -3.59440      0
4     0.3292   -4.4552    4.5711 -0.98880      1
5     4.3684    9.6718   -3.9606 -3.16250      0
6     3.5912    3.0126    0.7287 -0.56470      0
7     2.0922   -6.8100    8.9496 -0.83740      1
8     3.2032   -5.0882    2.0840 -0.06458      1
9     4.0030    0.7156   -0.0335 -0.82900      0
10    2.2285    4.1261   -0.1017 -0.04300      1
11    3.5770    2.9799    0.8892 -0.20240      0

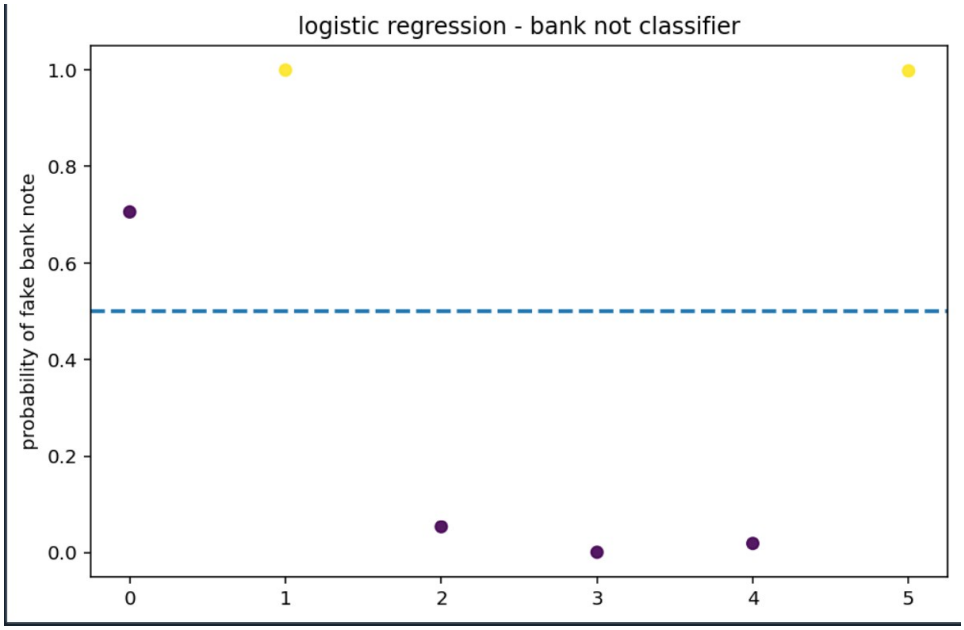
Intercept: [-2.09969975]

Coefficient: [[-0.4775123  -0.41475505  0.47663628 -0.14560022]]

Enter the new details:
----------------------
Enter variance:0.3292
Enter skewness:-4.4552
Enter curtosis:4.5711
Enter entropy:-0.98880

values 1.9136414783144025
Fake Note Probability: 0.8714276965527776

ZERO NOTE
Probability of FAKE NOTE: 0.12857230344722237
Probability of ZERO NOTE: 0.8714276965527776
PS C:\xampp\htdocs\hmsm>
```



RESULT:

Thus, the given program was successfully implemented using Python, and the required output was obtained successfully.

Ex. No: 05(a) Date:	K-Nearest Neighbors Algorithm	

AIM:

To implement the K-Nearest Neighbors (KNN) classification algorithm using Python to classify breast cancer data into two classes.

ALGORITHM:

Step 1: Import the required libraries and breast cancer dataset.

Step 2: Load the breast cancer dataset and separate input features (X) and target values (y).

Step 3: Display the shape of the data and target class names.

Step 4: Split the dataset into training and testing data.

Step 5: Scale the data using MinMaxScaler.

Step 6: Create the KNN classifier with 21 neighbors.

Step 7: Train the KNN model using the training data.

Step 8: Predict the classes using the testing data.

Step 9: Calculate the accuracy score.

Step 10: Display the confusion matrix.

PROGRAM:

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score,confusion_matrix,ConfusionMatrixDisplay
import matplotlib.pyplot as plt

cancer=load_breast_cancer()
x=cancer.data
y=cancer.target
print(f"Data Shape:{x.shape}")
print(f"Columns {cancer.target_names}")

x_train,x_test,y_train,y_test=train_test_split(
    x,
    y,
    train_size=100,
    random_state=47,
    stratify=y
)

scaler=MinMaxScaler()
x_train=scaler.fit_transform(x_train)
```

```
x_test=scaler.transform(x_test)


knn=KNeighborsClassifier(n_neighbors=21)
knn.fit(x_train,y_train)
y_pred=knn.predict(x_test)


print("\nAccuracy Score:")
print(accuracy_score(y_test,y_pred))
print("\nConfusion Matrix")
print(confusion_matrix(y_test,y_pred))


cm_knn = confusion_matrix(y_test, y_pred)


disp = ConfusionMatrixDisplay(
    confusion_matrix=cm_knn,
    display_labels=cancer.target_names
)

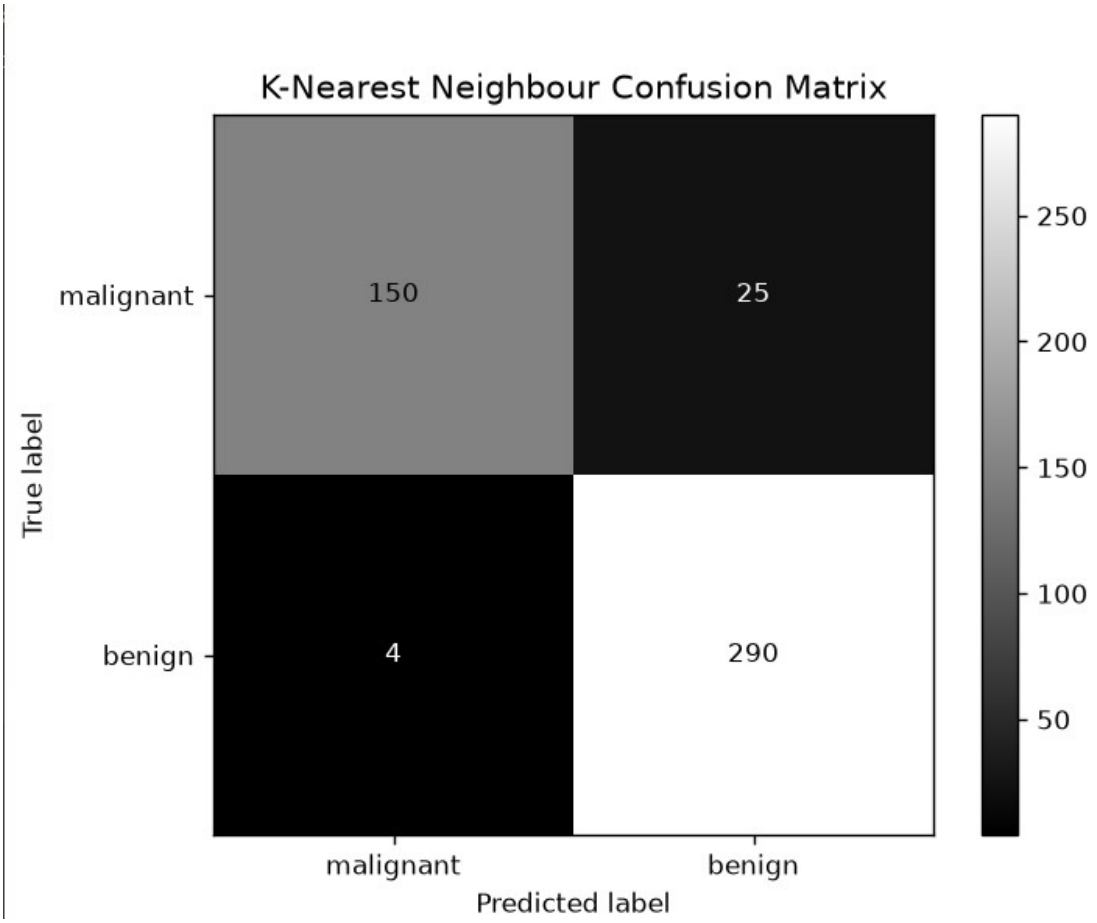

disp.plot(cmap="grey")
plt.title("K-Nearest Neighbour Confusion Matrix")
plt.show()
```

OUTPUT:

```
Data Shape:(569, 30)
Columns ['malignant' 'benign']
```

```
Accuracy Score:
0.9381663113006397

Confusion Matrix
[[150  25]
 [  4 290]]
```



RESULT:

Thus, the given program was successfully implemented using Python, and the required output was obtained successfully.

Ex. No: 05 (b) Date:	Naive Bayes Classification	

AIM:

To implement Gaussian Naive Bayes classification using Python to classify breast cancer data into Malignant and Benign classes.

ALGORITHM:

Step 1: Import the required libraries and breast cancer dataset.

Step 2: Load the breast cancer dataset and separate input features (X) and target values (y).

Step 3: Split the dataset into training and testing data.

Step 4: Create and train the Gaussian Naive Bayes model.

Step 5: Predict the classes using the test data.

Step 6: Calculate and display the accuracy score.

Step 7: Display the confusion matrix and classification report.

Step 8: Predict the class of the first test sample.

Step 9: Display the malignant and benign probabilities.

Step 10: Plot the confusion matrix using Matplotlib.

PROGRAM:

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score,
confusion_matrix,classification_report,ConfusionMatrixDisplay
import matplotlib.pyplot as plt


#! Step 1: Load Dataset
cancer = load_breast_cancer()
X = cancer.data
y = cancer.target


#! Step 2: Split Data
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=100,
    random_state=42,
    stratify=y
)


#! Step 3: Create and Train Model
nb = GaussianNB()
nb.fit(X_train, y_train)
```

```
#! Step 4: Predict Test Data
nb_pred = nb.predict(X_test)

#! Step 5: Accuracy
print("\nAccuracy:")
print(accuracy_score(y_test, nb_pred))

#! Step 6: Confusion Matrix
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, nb_pred))

#! Step 7: Classification Report
print("\nClassification Report:")
print(classification_report(
    y_test,
    nb_pred,
    target_names=cancer.target_names
))

#! Step 8: Prediction for First Test Sample
print("\nPrediction for First Test Sample")

prediction = nb.predict([X_test[0]])
probability = nb.predict_proba([X_test[0]])
```

```
if prediction[0] == 0:
    print("Diagnosis : Malignant")
else:
    print("Diagnosis : Benign")

print("Malignant Probability :", probability[0][0])
print("Benign Probability :", probability[0][1])

#! Step 9: Display Confusion Matrix
cm_nb = confusion_matrix(y_test, nb_pred)

disp = ConfusionMatrixDisplay(
    confusion_matrix=cm_nb,
    display_labels=cancer.target_names
)

disp.plot(cmap="Blues")
plt.title("Gaussian Naive Bayes Confusion Matrix")
plt.show()
```

OUTPUT:

```
Accuracy:
0.93

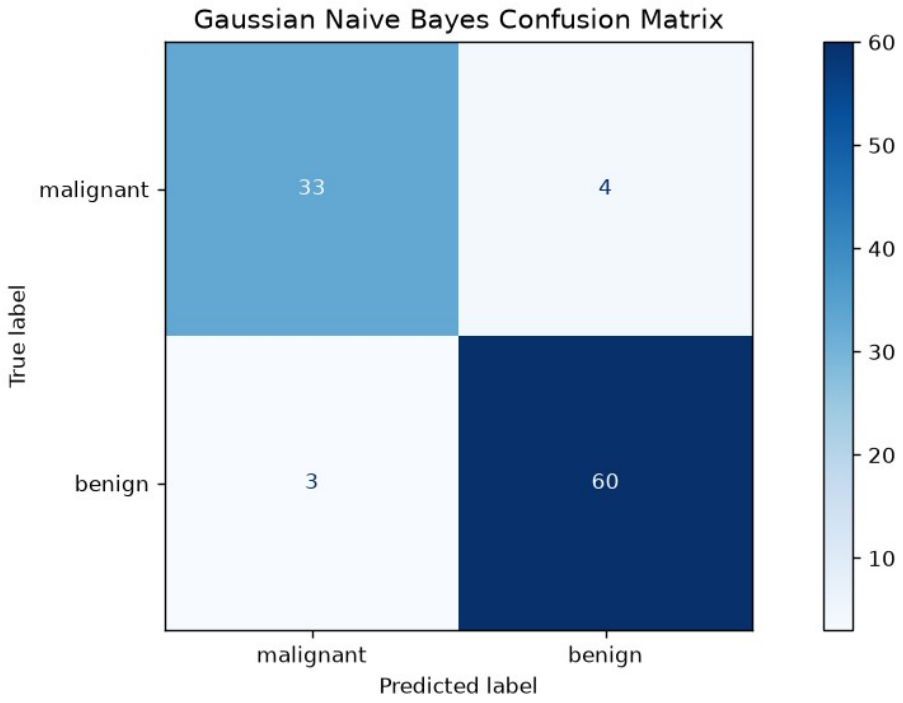
Confusion Matrix:
[[33  4]
 [ 3 60]]

Classification Report:
              precision    recall  f1-score   support

   malignant       0.92      0.89      0.90        37
      benign       0.94      0.95      0.94        63

    accuracy                           0.93       100
   macro avg       0.93      0.92      0.92       100
weighted avg       0.93      0.93      0.93       100


Prediction for First Test Sample
Diagnosis : Benign
Malignant Probability : 1.6627880044471545e-14
Benign Probability : 0.9999999999999831
```



RESULT:

Thus, the given program was successfully implemented using Python, and the required output was obtained successfully.

Ex. No: 06 Date:	Decision Tree with Hyperparameter Tuning	

AIM:

To implement a Decision Tree Classification algorithm using Python on the Iris dataset and tune its hyperparameters to improve the model performance.

ALGORITHM:

Step 1: Import the required libraries and Iris dataset.

Step 2: Create a DataFrame with the Iris features and target values.

Step 3: Separate the input features (X) and target labels (y).

Step 4: Split the dataset into training and testing data.

Step 5: Create and train the Decision Tree Classifier.

Step 6: Predict the classes and display accuracy, confusion matrix and classification report.

Step 7: Plot the Decision Tree using Matplotlib.

Step 8: Define different hyperparameters such as criterion, splitter, max depth and max features.

Step 9: Use GridSearchCV to find the best hyperparameters through cross-validation.

Step 10: Train and evaluate the best Decision Tree model.

Step 11: Predict the class of a single test record and display its probability.

Step 12: Plot the best Decision Tree model.

PROGRAM:

```
import pandas as pd
import matplotlib.pyplot as pt
import warnings
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.tree import DecisionTreeClassifier
from sklearn import tree
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report


warnings.filterwarnings("ignore")
iris = load_iris()
df = pd.DataFrame(
    iris.data,
    columns=[
        "sepal_length",
        "sepal_width",
        "petal_length",
        "petal_width"
    ])
df["target"] = iris.target
print("\nFirst five values")
print(df.head())
print("\nShape of dataframe")
```

```
print(df.shape)


# Step 2: Features and Labels
x = df.drop("target", axis=1)
y = df["target"]
print("\nFEATURES")
print(x.head())
print("\nLABELS")
print(y.head())


# Step 3: Split Dataset
x_train, x_test, y_train, y_test = train_test_split(
    x,
    y,
    test_size=0.20,
    random_state=47
)


print("\nInput training data")
print(x_train.head())
print("\nInput testing data")
print(x_test.head())
print("\nOutput training data")
print(y_train.head())
print("\nOutput testing data")
```

```
print(y_test.head())


print("\nInput training data shape")
print(x_train.shape)


print("\nInput testing data shape")
print(x_test.shape)


# Step 4: Train Decision Tree Model
model = DecisionTreeClassifier(random_state=0)


model.fit(x_train, y_train)


y_pred = model.predict(x_test)


print("\nAccuracy score:")
print(accuracy_score(y_test, y_pred))


print("Confusion matrix:")
print(confusion_matrix(y_test, y_pred))


print("\nClassification report:")
print(classification_report(
    y_test,
    y_pred,
```

```
    target_names=iris.target_names
))

# Step 5: Plot Decision Tree
pt.figure(figsize=(12, 8))
tree.plot_tree(
    model,
    feature_names=x.columns,
    class_names=iris.target_names,
    filled=True
)
pt.title("Decision Tree - Iris Dataset")
pt.show()

# Step 6: Hyperparameter Tuning
param = {
    "criterion": [
        "gini",
        "entropy",
        "log_loss"
    ],
    "splitter": [
        "best",
        "random"
    ],
```

```
    "max_depth": [1,2,3,4,5]
    "max_features": [
        None,
        "sqrt",
        "log2"
    ]
}
grid = GridSearchCV(
    estimator=DecisionTreeClassifier(
        random_state=0
    ),
    param_grid=param,
    cv=5,
    scoring="accuracy"
)

grid.fit(x_train, y_train)

print()

print("Best Hyperparameters")
print("Best parameters:")
print(grid.best_params_)

print("\nCross validation:")
```

```
print(grid.best_score_)


# Step 7: Best Model Prediction
best_model = grid.best_estimator_
y_pred = best_model.predict(x_test)
print("\nFinal test accuracy:")
print(accuracy_score(y_test, y_pred))
print("\nConfusion matrix:")
print(confusion_matrix(y_test, y_pred))


print("\nFinal classification report:")
print(classification_report(
    y_test,
    y_pred,
    target_names=iris.target_names
))


# Step 8: Single Record Prediction
first_record = x_test.iloc[[0]]


actual = y_test.iloc[0]


prediction = best_model.predict(first_record)


print()
```

```
print("Features")
print(first_record)

print("Actual class")
print(iris.target_names[actual])

print("Predicted class")
print(iris.target_names[prediction[0]])

# Step 9: Prediction Probability
probability = best_model.predict_proba(first_record)

print("\nPrediction probability:")

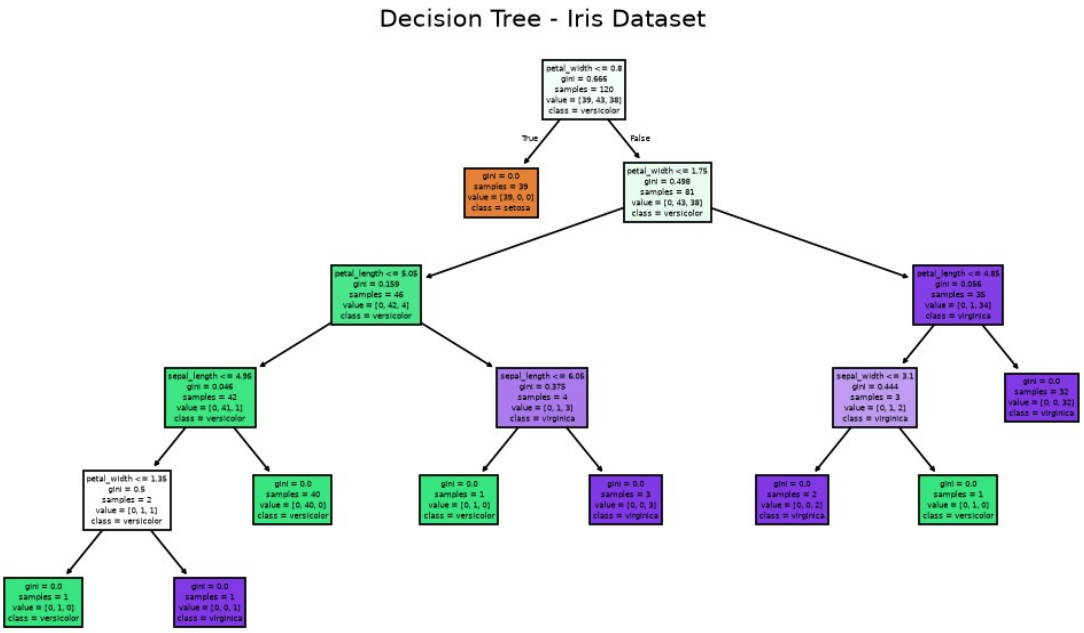
for i in range(len(iris.target_names)):
    print(
        iris.target_names[i],
        ":",
        probability[0][i]
    )

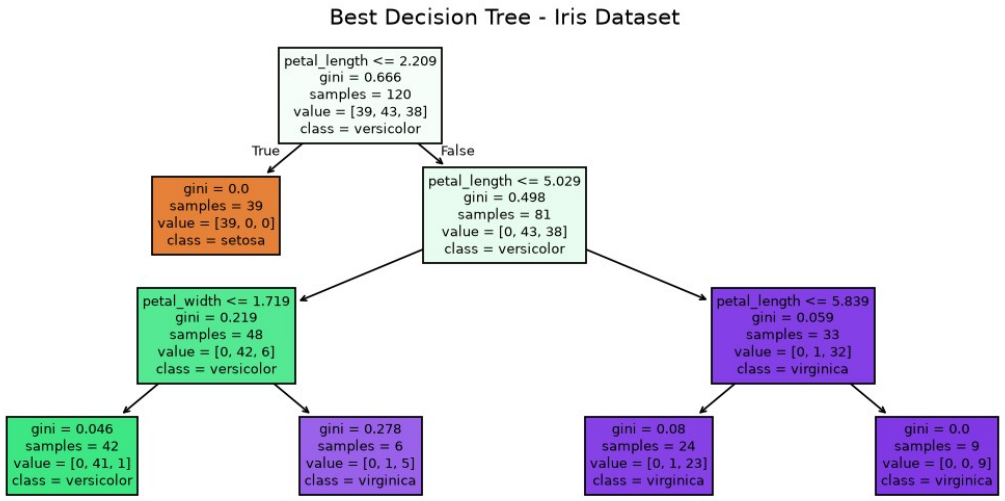
# Step 10: Plot Best Decision Tree
pt.figure(figsize=(12, 8))
```

```
tree.plot_tree(
    best_model,
    feature_names=x.columns,
    class_names=iris.target_names,
    filled=True
)

pt.title("Best Decision Tree - Iris Dataset")
pt.show()
```

OUTPUT:


Decision Tree - Iris Dataset
petal_width <= 0.8
gini = 0.666
samples = 120
value = [39, 43, 38]
class = versicolor
True
False
gini = 0.0
samples = 39
value = [39, 0, 0]
class = setosa
petal_width <= 1.75
gini = 0.498
samples = 81
value = [0, 43, 38]
class = versicolor
petal_length <= 5.05
gini = 0.159
samples = 46
value = [0, 42, 4]
class = versicolor
petal_length <= 4.85
gini = 0.056
samples = 35
value = [0, 1, 34]
class = virginica
sepal_length <= 4.96
gini = 0.046
samples = 42
value = [0, 41, 1]
class = versicolor
sepal_length <= 6.05
gini = 0.375
samples = 4
value = [0, 1, 3]
class = virginica
sepal_width <= 3.1
gini = 0.444
samples = 3
value = [0, 1, 2]
class = virginica
gini = 0.0
samples = 32
value = [0, 0, 32]
class = virginica
petal_width <= 1.35
gini = 0.5
samples = 2
value = [0, 1, 1]
class = versicolor
gini = 0.0
samples = 40
value = [0, 40, 0]
class = versicolor
gini = 0.0
samples = 1
value = [0, 1, 0]
class = versicolor
gini = 0.0
samples = 3
value = [0, 0, 3]
class = virginica
gini = 0.0
samples = 2
value = [0, 0, 2]
class = virginica
gini = 0.0
samples = 1
value = [0, 1, 0]
class = versicolor
gini = 0.0
samples = 1
value = [0, 1, 0]
class = versicolor
gini = 0.0
samples = 1
value = [0, 0, 1]
class = virginica


Best Decision Tree - Iris Dataset
petal_length <= 2.209
gini = 0.666
samples = 120
value = [39, 43, 38]
class = versicolor
True
False
gini = 0.0
samples = 39
value = [39, 0, 0]
class = setosa
petal_length <= 5.029
gini = 0.498
samples = 81
value = [0, 43, 38]
class = versicolor
petal_width <= 1.719
gini = 0.219
samples = 48
value = [0, 42, 6]
class = versicolor
petal_length <= 5.839
gini = 0.059
samples = 33
value = [0, 1, 32]
class = virginica
gini = 0.046
samples = 42
value = [0, 41, 1]
class = versicolor
gini = 0.278
samples = 6
value = [0, 1, 5]
class = virginica
gini = 0.08
samples = 24
value = [0, 1, 23]
class = virginica
gini = 0.0
samples = 9
value = [0, 0, 9]
class = virginica

```
Best parameters:
{'criterion': 'gini', 'max_depth': 3, 'max_features': None, 'splitter': 'random'}

Cross validation:
0.975

Final test accuracy:
0.9666666666666667

Confusion matrix:
[[11  0  0]
 [ 0  7  0]
 [ 0  1 11]]

Final classification report:
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        11
  versicolor       0.88      1.00      0.93         7
   virginica       1.00      0.92      0.96        12

    accuracy                           0.97        30
   macro avg       0.96      0.97      0.96        30
weighted avg       0.97      0.97      0.97        30


Features
    sepal_length  sepal_width  petal_length  petal_width
10           5.4          3.7           1.5          0.2
Actual class
setosa
Predicted class
setosa

Prediction probability:
setosa : 1.0
versicolor : 0.0
virginica : 0.0
```

RESULT:

Thus, the given program was successfully implemented using Python, and the required output was obtained successfully.

Ex. No: 07 (a) Date:	K-Means Clustering Using Mathematical Functions	

AIM:

To implement the K-Means Clustering algorithm manually using Python and NumPy to group data points into 3 clusters.

ALGORITHM:

Step 1: Import NumPy and Matplotlib libraries.

Step 2: Create a KMeans class with the number of clusters and maximum iterations.

Step 3: Generate 100 random data points using NumPy.

Step 4: Select random points as initial centroids.

Step 5: Calculate the distance between data points and centroids.

Step 6: Assign each data point to the nearest centroid.

Step 7: Calculate new centroids using the mean of each cluster.

Step 8: Repeat the process until the centroids stop changing or maximum iterations are reached.

Step 9: Assign the final clusters to the data points.

Step 10: Plot the data points and centroids using Matplotlib.

PROGRAM:

```
import numpy as np
import matplotlib.pyplot as plt


class KMeans:
    def __init__(self, n_clusters, max_iters=100):
        self.n_clusters = n_clusters
        self.max_iters = max_iters


    def fit(self, X):
        centroids_idx = np.random.choice(
            X.shape[0],
            size=self.n_clusters,
            replace=False
        )


        self.centroids = X[centroids_idx]


        for _ in range(self.max_iters):
            labels = self.assign_clusters(X)


            new_centroids = np.array([
                X[labels == k].mean(axis=0)
                for k in range(self.n_clusters)
            ])
```

```
            if np.allclose(self.centroids, new_centroids):
                break

            self.centroids = new_centroids

    def assign_clusters(self, X):
        distances = np.linalg.norm(
            X[:, np.newaxis] - self.centroids,
            axis=2
        )

        return np.argmin(distances, axis=1)


if __name__ == "__main__":
    np.random.seed(42)

    X = np.random.randn(100, 2)

    kmeans = KMeans(n_clusters=3)
    kmeans.fit(X)

    labels = kmeans.assign_clusters(X)
```

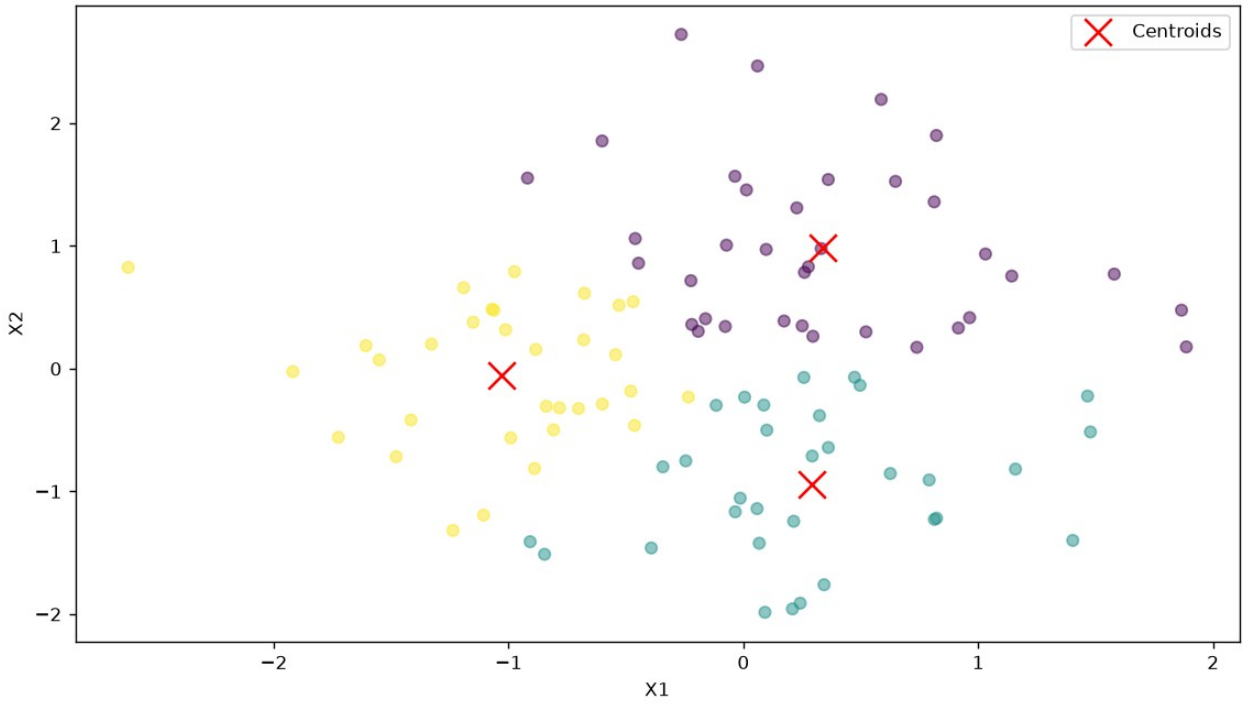
```
plt.figure(figsize=(8, 6))


plt.scatter(
    X[:, 0],
    X[:, 1],
    c=labels,
    cmap="viridis",
    alpha=0.5
)


plt.scatter(
    kmeans.centroids[:, 0],
    kmeans.centroids[:, 1],
    marker="x",
    c="red",
    s=200,
    label="Centroids"
)


plt.xlabel("X1")
plt.ylabel("X2")
plt.legend()
plt.show()
```

OUTPUT:



RESULT:

Thus, the given program was successfully implemented using Python, and the required output was obtained successfully.

Ex. No: 07 (b) Date:	K-Means Clustering using Python Library	

AIM:

To implement K-Means Clustering using the Scikit-learn library in Python to group data points into 3 clusters.

ALGORITHM:

Step 1: Import NumPy, Matplotlib and KMeans libraries.

Step 2: Generate 100 random data points using NumPy.

Step 3: Create a K-Means model with 3 clusters.

Step 4: Set the maximum iterations and random state.

Step 5: Train the K-Means model using the data.

Step 6: Get the cluster labels and centroids.

Step 7: Plot the data points based on their clusters.

Step 8: Display the centroids using an X marker.

Step 9: Add labels, title and legend to the graph.

Step 10: Display the K-Means clustering graph.

PROGRAM:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans


np.random.seed(42)
X = np.random.rand(100, 2)
kmeans = KMeans(n_clusters=3, max_iter=100, random_state=42, n_init=10)
kmeans.fit(X)
labels = kmeans.labels_
centroids = kmeans.cluster_centers_

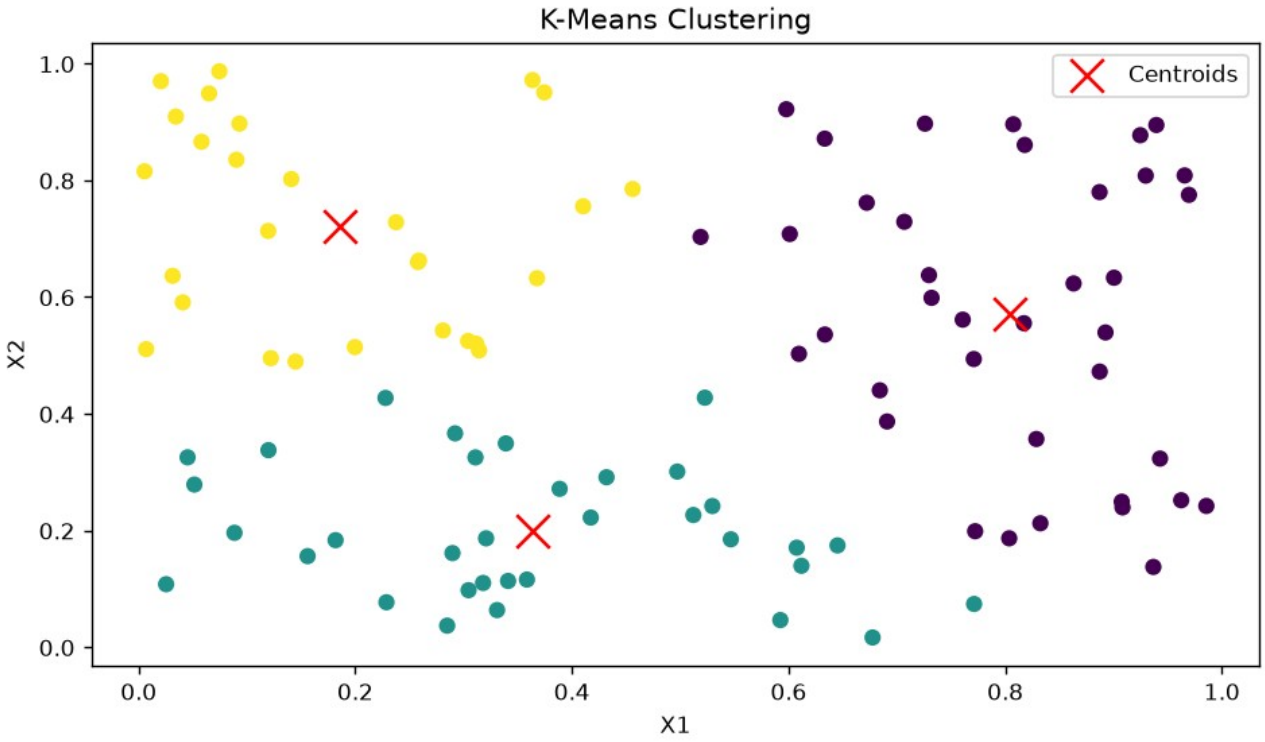

plt.figure()


plt.scatter(X[:, 0], X[:, 1], c=labels, cmap="viridis")


plt.scatter(centroids[:, 0], centroids[:, 1], marker="x", s=200, c="red", label="Centroids")


plt.xlabel("X1")
plt.ylabel("X2")
plt.title("K-Means Clustering")
plt.legend()
plt.show()
```

OUTPUT:



RESULT:

Thus, the given program was successfully implemented using Python, and the required output was obtained successfully.

Ex. No: 08 Date:	CNN Using Tensorflow	

AIM:

To implement a Convolutional Neural Network (CNN) using TensorFlow to recognize handwritten digits from the MNIST dataset.

ALGORITHM:

Step 1: Import TensorFlow and required libraries.

Step 2: Load the MNIST handwritten digit dataset.

Step 3: Normalize the image values between 0 and 1.

Step 4: Reshape the images into 28 × 28 × 1 format.

Step 5: Create a CNN model using convolution, pooling and dense layers.

Step 6: Compile the model using the Adam optimizer and loss function.

Step 7: Train the model for 5 epochs using training data.

Step 8: Evaluate the model using test data.

Step 9: Display the test accuracy.

Step 10: Predict the digit of the first test image.

Step 11: Display the actual digit and predicted digit.

PROGRAM:

```
from tensorflow.keras import datasets, layers, models

(x_train,y_train),(x_test,y_test)=datasets.mnist.load_data()

x_train=x_train/255.0
x_test=x_test/255.0

x_train=x_train.reshape(-1,28,28,1)
x_test=x_test.reshape(-1,28,28,1)

model=models.Sequential([
    layers.Conv2D(32,(3,3),activation='relu',input_shape=(28,28,1)),
    layers.MaxPooling2D((2,2)),
    layers.Conv2D(64,(3,3),activation='relu'),
    layers.MaxPooling2D((2,2)),
    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax')
])

model.compile(optimizer='adam',loss='sparse_categorical_crossentropy',metrics=['accuracy'])

model.fit(x_train,y_train,epochs=5,validation_split=0.1)
```

```
test_loss,test_accuracy=model.evaluate(x_test,y_test)
print("test accuracy:",test_accuracy)

prediction=model.predict(x_test[0:1])
predict_class=prediction.argmax()

print("actual digit:",y_test[0])
print("prediction digit:",predict_class)
```

OUTPUT:

```
In [20]: runfile('C:/Users/Admin/Desktop/AMLLAB/labpgms/cnn_digit1.py', wdir='C:/Users/
Admin/Desktop/AMLLAB/labpgms')
Model: "sequential_11"
```

Layer (type)	Output Shape	Param #
conv2d_2 (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d_2 (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_3 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_3 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten_1 (Flatten)	(None, 1600)	0
dense_30 (Dense)	(None, 64)	102,464
dense_31 (Dense)	(None, 10)	650

```
 Total params: 121,930 (476.29 KB)
 Trainable params: 121,930 (476.29 KB)
 Non-trainable params: 0 (0.00 B)
Epoch 1/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 10s 6ms/step - accuracy: 0.9536 - loss: 0.1503 -
val_accuracy: 0.9818 - val_loss: 0.0595
Epoch 2/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 13s 8ms/step - accuracy: 0.9849 - loss: 0.0471 -
val_accuracy: 0.9880 - val_loss: 0.0405
Epoch 3/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 12s 7ms/step - accuracy: 0.9896 - loss: 0.0327 -
val_accuracy: 0.9900 - val_loss: 0.0364
Epoch 4/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 12s 7ms/step - accuracy: 0.9924 - loss: 0.0245 -
val_accuracy: 0.9892 - val_loss: 0.0392
Epoch 5/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 13s 8ms/step - accuracy: 0.9940 - loss: 0.0178 -
val_accuracy: 0.9890 - val_loss: 0.0425
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9903 - loss: 0.0301

Test Accuracy: 0.9902999997138977
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 73ms/step
Actual Digit: 7
Predicted Digit: 7
```

RESULT:

Thus, the given program was successfully implemented using Python, and the required output was obtained successfully.

Ex. No: 09 Date:	Auto Encoders Using Tensorflow	

AIM:

To implement an Autoencoder using TensorFlow to remove noise from handwritten MNIST images and reconstruct clear images.

ALGORITHM:

Step 1: Import TensorFlow, NumPy, Matplotlib and required libraries.

Step 2: Load the MNIST dataset and display the training and testing data shape.

Step 3: Normalize the image values between 0 and 1.

Step 4: Add random noise to the training and testing images.

Step 5: Clip the noisy image values between 0 and 1.

Step 6: Reshape the images into 784 input values.

Step 7: Create an Autoencoder with encoder and decoder layers.

Step 8: Compile the model using Adam optimizer and binary cross-entropy loss.

Step 9: Train the Autoencoder using noisy images as input and original images as output.

Step 10: Reconstruct the noisy test images using the trained model.

Step 11: Display original, noisy and denoised images.

Step 12: Plot the training and validation loss graph.

PROGRAM:

```
import tensorflow as tf
import matplotlib.pyplot as plt
import numpy as np
from tensorflow.keras import layers, models


(x_train, _), (x_test, _) = tf.keras.datasets.mnist.load_data()
print("training data shape:", x_train.shape)
print("testing data shape:", x_test.shape)


x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0


noise_factor = 0.5
x_train_noisy = x_train + noise_factor * np.random.normal(
    loc=0.0,
    scale=1.0,
    size=x_train.shape
)
x_test_noisy = x_test + noise_factor * np.random.normal(
    loc=0.0,
    scale=1.0,
    size=x_test.shape
)
```

```
x_train_noisy = np.clip(x_train_noisy, 0., 1.)
x_test_noisy = np.clip(x_test_noisy, 0., 1.)

x_train = x_train.reshape(len(x_train), 784)
x_test = x_test.reshape(len(x_test), 784)

x_train_noisy = x_train_noisy.reshape(len(x_train_noisy), 784)
x_test_noisy = x_test_noisy.reshape(len(x_test_noisy), 784)

input_layer = layers.Input(shape=(784,))

encoded = layers.Dense(128, activation='relu')(input_layer)
encoded = layers.Dense(64, activation='relu')(encoded)
encoded = layers.Dense(32, activation='relu')(encoded)

decoded = layers.Dense(64, activation='relu')(encoded)
decoded = layers.Dense(128, activation='relu')(decoded)
decoded = layers.Dense(784, activation='sigmoid')(decoded)

autoencoder = models.Model(input_layer, decoded)

autoencoder.compile(
    optimizer="adam",
    loss="binary_crossentropy"
)
```

```
history = autoencoder.fit(
    x_train_noisy,
    x_train,
    epochs=10,
    batch_size=256,
    shuffle=True,
    validation_data=(x_test_noisy, x_test)
)

reconstructed = autoencoder.predict(x_test_noisy)

n = 10
plt.figure(figsize=(20, 6))

for i in range(n):
    ax = plt.subplot(3, n, i + 1)
    plt.imshow(x_test[i].reshape(28, 28), cmap="gray")
    plt.axis("off")
    ax.set_title("original")

    ax = plt.subplot(3, n, i + 1 + n)
    plt.imshow(x_test_noisy[i].reshape(28, 28), cmap="gray")
    plt.axis("off")
    ax.set_title("noisy")
```

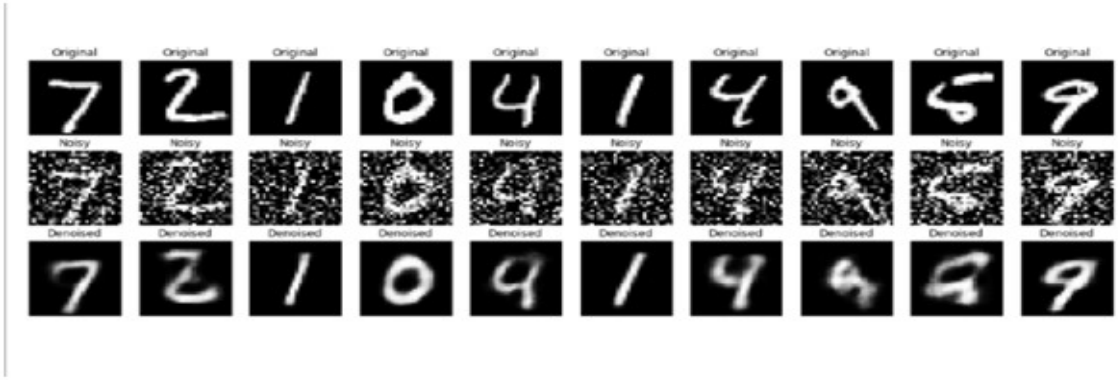
```
    ax = plt.subplot(3, n, i + 1 + 2 * n)
    plt.imshow(reconstructed[i].reshape(28, 28), cmap="gray")
    plt.axis("off")
    ax.set_title("denoised")

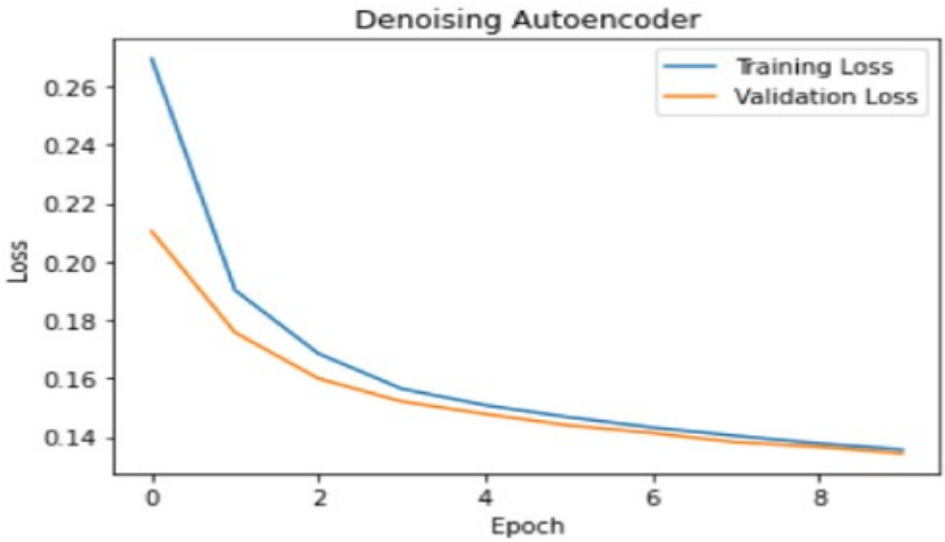
plt.show()

plt.plot(history.history['loss'], label="training loss")
plt.plot(history.history['val_loss'], label="validation loss")

plt.legend()
plt.show()
```

OUTPUT:





RESULT:

Thus, the given program was successfully implemented using Python, and the required output was obtained successfully.

Ex. No: 10 Date:	Long Short-Term Memory Using Tensorflow	

AIM:

To implement Long Short-Term Memory (LSTM) using TensorFlow to predict the next temperature value based on previous temperature values.

ALGORITHM:

Step 1: Import NumPy, TensorFlow and required LSTM libraries.

Step 2: Create a temperature dataset.

Step 3: Scale the temperature values between 0 and 1.

Step 4: Set the sequence length to 3.

Step 5: Create input sequences using 3 previous temperatures and the next temperature as output.

Step 6: Reshape the input data into LSTM format.

Step 7: Create an LSTM model with 50 units and a Dense output layer.

Step 8: Compile the model using Adam optimizer and MSE loss.

Step 9: Train the model for 500 epochs.

Step 10: Give the last 3 temperatures (42, 43, 44) as input.

Step 11: Predict the next temperature using the trained LSTM model.

Step 12: Convert the predicted value back to the original temperature scale and display the result.

PROGRAM:

```
import numpy as np
import tensorflow as tf
from tensorflow.keras import Sequential, Input
from tensorflow.keras.layers import LSTM, Dense


temp=np.array([
    30,31,32,33,34,
    35,36,37,38,39,
    40,41,42,43,44],dtype=float)


minimum=temp.min()
maximum=temp.max()


temp_scaled=(temp-minimum)/(maximum-minimum)


sequence_length=3
x=[]
y=[]


for i in range(len(temp_scaled)-sequence_length):
    x.append(temp_scaled[i:i+sequence_length])
    y.append(temp_scaled[i+sequence_length])


x=np.array(x)
```

```
y=np.array(y)

x=x.reshape(x.shape[0],x.shape[1],1)

print("input shape:",x.shape)
print("output shape:",y.shape)

model=Sequential([
    Input(shape=(sequence_length,1)),
    LSTM(50),
    Dense(1)
    ])

model.compile(
    optimizer="adam",
    loss="mse"
    )

model.fit(x,y,epochs=500,verbose=0)

test_sequence=np.array([42,43,44], dtype=float)
test_scaled=(test_sequence-minimum)/(maximum-minimum)
test_scaled=test_scaled.reshape(1,sequence_length,1)

prediction_scaled=model.predict(
```

```
    test_scaled,
    verbose=0)

prediction=(
    prediction_scaled[0][0]*(maximum-minimum)+minimum)

print("\n input temperature:",[42,43,44])
print("predicted next temperature:",prediction)
```

OUTPUT:

```
Input shape: (12, 3, 1)
Output shape: (12,)

Input temperature: [42, 43, 44]
Predicted next temperature: 45.36069869995117
```

RESULT:

Thus, the given program was successfully implemented using Python, and the required output was obtained successfully.